Davante

8

PROGRAMACIÓN

Gestión de clases y objetos

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Objetos 5

/ 3. Ciclo de vida de un objeto 5

/ 4. Caso práctico 1: "Rapidez o buen diseño" 7

/ 5. Almacenamiento en memoria 7

/ 6. Referencias 8

6.1. Variables referencia 8

6.2. Referencia nula 9

/ 7. Instanciación de objetos 9

/ 8. Recolector de basura 11

/ 9. Métodos 12

/ 10. Ámbito de las variables 13

/ 11. Ocultación de atributos 15

/ 12. Caso práctico 2: "Shadowing de variables" 16

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 13. Paso de parámetros y método toString() 17

/ 14. Documentación de métodos 18

/ 15. Resumen y resolución del caso práctico de la unidad 19

/ 16. Bibliografía 20

OBJETIVOS

Identificar las características de un objeto.

Conocer el ciclo de vida de un objeto.

Crear y utilizar objetos.

Comprender el almacenamiento en memoria de los objetos.

Utilizar atributos y métodos de los objetos.

Diferenciar los distintos ámbitos de variables y su visibilidad.

Diferenciar entre paso por valor y paso por referencia.

Conocer la funcionalidad del método toString.

Documentar los métodos de una clase con Doxygen.

/ 1. Introducción y contextualización práctica

En este tema hablaremos sobre los objetos, sus características, su ciclo de vida y su almacenamiento en memoria.

También se verá cómo obtener los datos de los objetos y cómo mostrar correctamente nuestros objetos a los usuarios teniendo en cuenta los distintos ámbitos de visibilidad.

Se va a profundizar en los métodos, viendo con detenimiento en qué consiste el paso por valor y el paso por referencia. También nos adentraremos en el método toString.

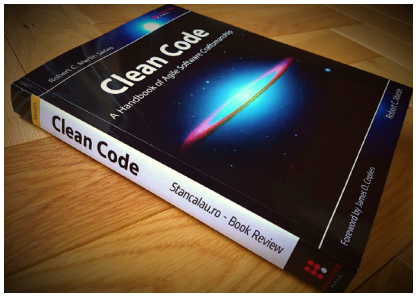

Fig.1. Clean Code.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Divide y vencerás"

https://on.soundcloud.com/7ftYHDpwiEzZxhB79

/ 2. Objetos

Ya sabemos que los objetos en POO son representaciones de entidades del mundo real en nuestros programas, de manera que los objetos físicos son fáciles de representar. Sin embargo, **algunos conceptos no son tan fáciles de identificar**. Para ello, el procedimiento para saber si algo puede ser candidato a objeto es analizar lo siguiente:

- ¿Tiene características propias?
- ¿Puede realizar acciones propias?

Así, **un objeto viene definido por su identidad, estado y un comportamiento asociado**:

- **Identidad**: Esta propiedad nos va a permitir **distinguir un objeto de otro del mismo tipo**, de forma que, aunque dos objetos del mismo tipo tengan los mismos valores en sus atributos, serán distintos entre sí, es decir, serán independientes los unos de los otros. En Java esto se implementa mediante direcciones de memoria.
- **Estado**: Esta propiedad viene definida por todos los atributos de los objetos, es decir, se compone de todas las variables u objetos que componen un objeto, las denominadas **variables miembro o variables de objeto**. Estas pueden ser de los tipos primitivos (*int, double, char, boolean*...) o pueden ser objetos de otras clases.

  Por ejemplo, si tenemos una clase Vehículo como la que se muestra en la figura 2, podremos crear distintas instancias de la clase con distintos estados.

Fig.2. Objetos Coche.

- **Comportamiento**: Esta propiedad viene definida por todas las **acciones que puede realizar un objeto**, es decir, por los métodos del objeto, desde los constructores hasta los métodos definidos por el programador.

Podemos considerar, por tanto, que un objeto engloba tanto propiedades como métodos. Como ya sabemos, los datos están encapsulados dentro del objeto. Esto se consigue declarándolos con un ámbito privado, y la única forma de modificarlos es mediante sus métodos.

/ 3. Ciclo de vida de un objeto

Todos los programas que creemos en Java van a partir de una clase, 'la clase principal', que es el punto de entrada para cualquier programa en Java. Esta clase ejecutará un método principal denominado *main* que podemos ver cuando creamos un proyecto nuevo en NetBeans.

Todas las instrucciones que se ejecuten en el método main se realizarán de manera secuencial hasta finalizar el programa y terminar su ejecución.

Durante esa ejecución de instrucciones, se crearán distintas instancias de objetos que, de manera similar a un ser vivo, poseen un ciclo de vida: 'nacen', 'son utilizados', 'mueren'.

Por tanto, cualquier objeto pasa por las siguientes etapas:

- **Creación**: En esta fase, se crea el objeto utilizando la palabra clave '**new'** seguida del constructor de la clase. El objeto se asigna a una variable de **referencia** y se reserva espacio en la memoria para almacenar sus datos.

- **Inicialización**: Después de la creación, el objeto puede ser inicializado con valores predeterminados o mediante llamadas a métodos especiales (constructores). Durante esta etapa, **se asignan valores** a las variables de instancia del objeto.

- **Uso**: En esta fase, el objeto está en un estado activo y se utiliza para **realizar operaciones específicas** según el diseño de la clase a la que pertenece. Los métodos y variables de instancia del objeto pueden ser accedidos y modificados.

- **Finalización**: Cuando el objeto **ya no es necesario**, puede pasar por un proceso de finalización. Esto puede implicar liberar recursos, cerrar conexiones o cualquier otra limpieza necesaria para liberar la memoria o los recursos utilizados por el objeto.

- **Destrucción**: Una vez que el objeto ha sido finalizado, está listo para ser eliminado de la memoria. En Java, esto se hace de manera automática mediante el **recolector de basura** (*garbage collector*), que identifica objetos que ya no son accesibles y libera la memoria que ocupan.

¡! Sabías que...

La finalización se refiere a la ejecución del método finalize() de un objeto antes de que sea destruido por el recolector de basura, mientras que la destrucción es el proceso de eliminación real del objeto y la liberación de la memoria que ocupaba.

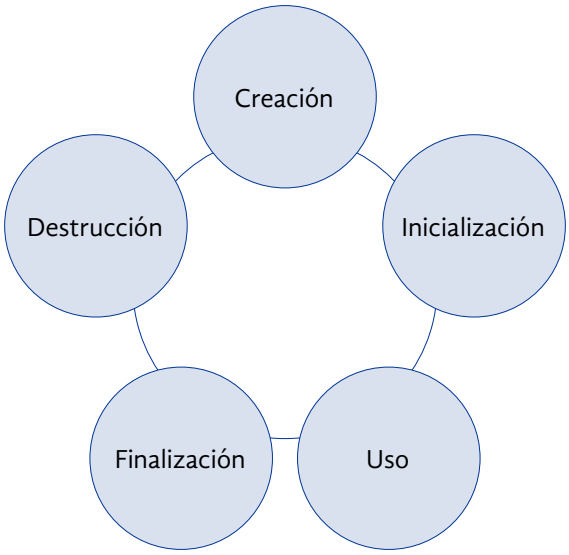

Fig.3. Ciclo de vida de un objeto.

/ 4. Caso práctico 1: "Rapidez o buen diseño"

Planteamiento: El equilibrio entre rapidez de desarrollo y buen diseño de *software* es fundamental para crear aplicaciones robustas y mantenibles cogiendo como base el principio SOLID.

Fig.4. SOLID.

Nudo: Se está realizando un ejercicio en el que hay que crear clases que representen lo necesario para la gestión de una clínica veterinaria. Hablando sobre la resolución del mismo, se plantea si lo mejor y más rápido sería guardar toda la información necesaria en una única clase, ya que, así, con la creación de un objeto, se podrá resolver el problema de forma rápida y fácil.

¿Qué piensas sobre ello? ¿Crees que es apropiado guardar toda la información que se necesita en una clase? ¿Así se resolverá el problema más rápido?

Desenlace: Al realizar un programa que utilice orientación a objetos, se deberá parar para pensar en el diseño de la solución. Todo buen diseño se basa en la creación de clases que representen fielmente las entidades del mundo real, por tanto, se obtendrán varias clases en cualquier proyecto. Meter en una única clase toda la información necesaria es una mala práctica, ya que no se tendrá la claridad que ofrecen las clases bien diseñadas ni la abstracción de la información que estas proveen.

Como conclusión, vale la pena dedicarle tiempo a pensar el diseño de la solución para conseguir un código mucho más sencillo de hacer y comprender.

/ 5. Almacenamiento en memoria

Cuando ejecutamos un programa, nos apoyamos en la memoria RAM del ordenador, es decir, **todos los objetos y variables que usemos estarán alojados en memoria durante la ejecución del mismo**. Todo este control lo realiza el sistema operativo, es decir, que es totalmente transparente para el usuario.

Durante la ejecución del programa se van a usar diferentes zonas de memoria, cada una de ellas con un cometido diferente. Estas son:

- *Callstack*: La **pila de llamadas** es una estructura dinámica de datos encargada del almacenamiento de información sobre las distintas subrutinas que mantiene el programa en ejecución. Al hacer una llamada a un método, se reserva un bloque memoria en la cima de la pila que permite almacenar los datos locales empleados en la función.

  La pila es una estructura LIFO (del inglés, *Last In, First Out*), en la que el último elemento que entró es el primero en salir. Cada vez que se llame a un método, se insertará una entrada en la pila.

Pueden existir varias pilas en ejecución de manera paralela si es multihilo, de manera que cada hilo tenga su propia pila de llamadas.

- *Heap*: El **área de datos** hace referencia a la zona de memoria donde se van a almacenar todas las variables empleadas para almacenar datos en ejecución, tanto primitivas como objetos. Cada vez que ejecutemos un new estaremos almacenando el objeto en heap. Esta zona de memoria es la encargada de almacenar las variables globales y estáticas.

  La siguiente figura 5 muestra ambos conceptos: la pila de llamadas y el área de datos.

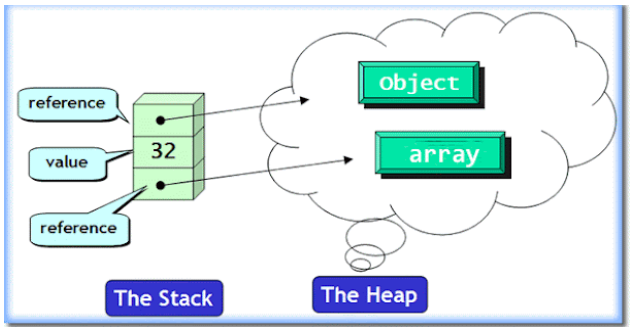

Fig.5. Stack y Heap.

- **Otras áreas de memoria: Existen dos áreas más:**

  » **El área de datos estáticos**: En esta zona de memoria es donde se van a almacenar las variables globales del programa y las variables estáticas.

  » **El área del código**: Esta es el área de memoria donde se van a almacenar las instrucciones que el programa debe ejecutar. Cuantas más líneas de código tenga el programa, más ocupará esta zona de memoria.

/ 6. Referencias

A la hora de trabajar con objetos, se realiza de igual manera que con las tablas, por medio de referencias, de ahí que **las tablas se consideren en Java un objeto** en sí.

La memoria del ordenador está formada por bloques de memoria consecutivos denominados **direcciones de memoria**. Estas emplean números hexadecimales dando lugar a un aspecto similar al siguiente: 3b240e44.

Por tanto, todos los datos almacenados en memoria ocuparán distintos bloques de memoria que serán identificados mediante una dirección de memoria. A esta se la denomina **referencia**. De manera que una referencia, en Java, es un puntero, un alias o un nombre a un objeto.

6.1. Variables referencia

Antes de construir un objeto es necesario declarar una **variable que sea de la misma clase que el objeto que vamos a crear**. Para ello se emplean las mismas normas que utilizamos al declarar un tipo primitivo.

```
TipoPrimitivonombreVariable;
Clase nombreObjeto;
```

La diferencia en ambas declaraciones está en que el tipo primitivo almacena un valor, mientras que la variable del tipo Clase almacena una posición en memoria.

De manera que, si queremos declarar un objeto llamado cocheFran de la clase Coche, el código sería el siguiente:

```
CochecocheFran;
```

6.2. Referencia nula

Una referencia de objeto puede tener el valor especial *null*, lo que indica que **no está referenciando ningún objeto en la memoria**. Al realizar la declaración de un objeto, por defecto tiene un valor null aunque también se le puede asignar.

```
CochecocheFran = null;
```

Intentar acceder a los miembros de un objeto con una referencia nula provocará una excepción NullPointerException. En la figura 6, la variable tiene una referencia nula

```
    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        Coche cocheFran;
variable cocheFran might not have been initialized
----
(Alt-Enter shows hints)
        cocheFran.acelerar();


    }
```

Fig.6. Error NullPointerException.

Audio 1. "Consecuencias de igualar un objeto a null"
https://bit.ly/2DwZqqU

/ 7. Instanciación de objetos

Ya hemos visto que los objetos se declaran como si fuesen variables. La sintaxis para la declaración de un objeto es la siguiente:

```
NombreClasenombreobjeto;
```

Una vez declarado el objeto, falta la parte más importante, su instanciación. Cuando instanciamos un objeto, estamos haciendo que el sistema operativo reserve memoria en heap para poder almacenarlo y, por lo tanto, que el objeto tenga un valor válido.

La **instanciación o creación de objetos** se hace mediante la sentencia new y su sintaxis es:

```
nombreobjeto = new NombreClase( [parámetros] );
```

Mediante esta sentencia, no solo se declara una variable del tipo de la clase que queremos crear, sino que se crea el objeto asignando un número consecutivo de bloques de memoria.

Observemos la figura 7.

```
6   package coches;
7
8   /**
9    *
10   * @author franciscojimenezmoral
11   */
12  public class Coches {
13
14      /**
15       * @param args the command line arguments
16       */
17      public static void main(String[] args) {
18          // TODO code application logic here
19          Coche cocheFran = null;
20          System.out.println(cocheFran);
21
22          cocheFran = new Coche();
23          System.out.println(cocheFran);
```

coches.Coches > main >

Output – Coches (run)

```
run:
null
coches.Coche@15db9742
BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig.7. Creación de objetos.

En esta figura 7 se puede comprobar el aspecto que tiene una referencia al objeto. En el primer caso, la variable cocheFran de tipo Coche con una referencia null, y en el segundo caso, la misma variable cocheFran con un espacio en memoria que muestra la referencia al primer bloque de memoria, en este caso Coche@**15db9742**. El resto de propiedades del objeto cocheFran se guardan en los bloques contiguos de memoria, como se puede observar en la siguiente tabla 1.

	15db9741	.....
Variable cocheFran →	**15db9742**	**marca: Volkswagen**
	15db9743	**modelo: T-Roc**
	15db9744	**matrícula: 8612KRZ**
	15db9745	.....

Tabla 1. Representación de un objeto en memoria.

Se puede dar el caso de crear una nueva variable de tipo Coche que almacene la misma referencia al objeto cocheFran, es decir, un objeto referenciado por dos variables (observemos la figura 8).

```
Coche cocheFran = new Coche("volkswagen",
"T-Roc","8612KRZ");

Coche karroza;

karroza = cocheFran;
```

A continuación podrás ver un recurso gráfico sobre el punto anterior:

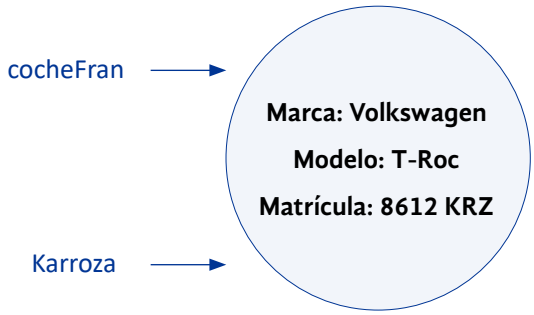

Fig.8. Objeto referenciado por dos variables.

/ 8. Recolector de basura

El recolector de basura (garbage collector) es una pieza clave del lenguaje Java. Es una grandísima ventaja frente a otros lenguajes que no lo poseen, como, por ejemplo, *C++*.

El recolector de basura es el **encargado de eliminar, de forma automática, todos aquellos objetos que ya no son referenciados por ninguna variable y que se quedan 'a la deriva' en la memoria del ordenador**. Para ello, el recolector de basura monitorea continuamente la memoria asignada a los objetos. Cuando detecta un objeto que ya no tiene referencias que lo apunten (es decir, el objeto se ha vuelto inaccesible), marca ese objeto como candidato para ser recolectado.

Su funcionamiento es el siguiente:

- Cada vez que instanciamos un objeto, este se va a crear en un espacio de memoria llamado heap, especialmente creado para dicho fin.
- Cuando hemos terminado de trabajar con dicho objeto, se queda sin una variable que lo referencie, pero no es eliminado del heap.
- Mientras tanto, cada cierto tiempo, antes de empezar a ejecutar un programa Java, notaremos que se tarda un poquito más de lo normal en ejecutar, esto nos indicará que se está ejecutando el recolector de basura, y es en ese instante cuando se localizarán del heap todos los objetos no referenciados, tras lo cual, se procederá a eliminarlos de la memoria uno a uno.

En la siguiente figura 9 se puede observar el funcionamiento:

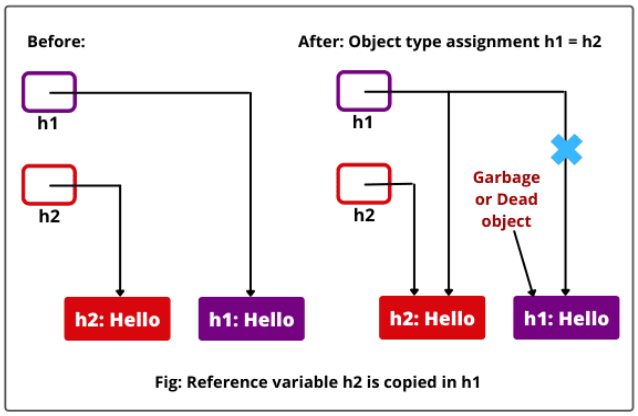

Fig.9. Objeto referenciado por dos variables.

En el ejemplo se puede observar cómo inicialmente los objetos h1 y h2 apuntan a dos espacios en memoria cuyo contador de referencias es 1 para ambos. El objeto h1 cambia su referencia a h2 y el valor previo de h1 deja de ser referenciado. En ese momento el contador de referencias del espacio de memoria que referencia h2 pasa a 2 mientras que el contador de referencias del espacio que apunta previamente h1 pasa a 0 y, por tanto, borrado por el garbage collector.

¡! Sabías que...

El recolector de basura puede utilizar varias técnicas de borrado de objetos no referenciados, como, por ejemplo, el borrado paralelo o el borrado G1.

/ 9. Métodos

Los métodos, junto a los atributos, son la otra parte que compone a las clases. Constituyen un elemento fundamental y, al igual que no tiene sentido una clase sin atributos, no tiene sentido una clase sin métodos.

Tenemos diferentes tipos de métodos en las clases en Java:

- **Constructores**: Son los encargados de inicializar los objetos. Pueden ser por defecto o con parámetros.
- **Observadores**: Su misión es consultar el valor de los atributos de las clases.
- **Modificadores**: Con estos se modifica el valor de los atributos de las clases.
- **Métodos 'personalizados'**: Son aquellos que tendremos que definir según las funcionalidades que necesitemos que ejecute nuestra clase.

A la hora de nombrarlos, se realiza según la notación CamelCase. Los métodos se aplican siempre a un objeto de la clase usando el **operador 'punto'**. Pueden llevar argumentos o parámetros entre paréntesis y no devolver nada o devolver un único valor de cualquier tipo.

¡! Sabías que...

El CamelCase es una convención de escritura que se utiliza comúnmente en la programación para formar nombres de variables, métodos y, en ocasiones, nombres de clases.

En el caso de que el método no devuelva ningún valor, irá precedido de la palabra reservada ***void***. En caso contrario, solo podrá devolver un único valor, mediante la instrucción ***return***.

A continuación vamos a desarrollar los métodos saludo(), nombreCompleto() y cumplirAños() de la clase Persona (observemos la figura 10):

```
public class Persona {
    String nombre;
    int edad;
    String apellido;

    public void saludo(String nombre){
        System.out.println("Saludos "+nombre);
    }

    public String nombreCompleto(){
        return nombre + " " +apellido;
    }

    public void cumplirAños(){
        edad ++;
        System.out.println("Has cumplido "+edad);
    }
}
```

Fig.10. Clase Persona.

Para comprobar su funcionamiento, vamos a crear un objeto de tipo Persona para probar sus métodos como muestra la figura 11.

```
12  public class Personas {
13
14      public static void main(String[] args) {
15          // TODO code application logic here
16
17          Persona fran = new Persona();
18          fran.nombre = "fran";
19          fran.apellido = "jimenez";
20          fran.edad = 32;
21
22          fran.saludo("Alumno de Medac");
23
24          System.out.println("El nombre completo es: "+fran.nombreCompleto());
25          fran.cumplirAños();
26          System.out.println("");
27      }
28  }
```

```
Output - Persona (run)
run:
Saludos Alumno de Medac
El nombre completo es: fran jimenez
Has cumplido 33

BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig.11. Prueba de métodos de la clase Persona.

/ 10. Ámbito de las variables

El ámbito de una variable se encarga de definir dónde puede emplearse la misma. Hasta ahora tenemos dos ámbitos en el uso de variables:

- **Dentro de un bloque de estructura de control**: Las variables definidas dentro de un *if-else*, *switch* o cualquier bucle se denominan **variables de bloque**.

- **Dentro de una función**: Son aquellas variables definidas dentro de una función denominadas **variables locales**.

Gracias a la POO, surge un nuevo ámbito:

- **Dentro de una clase**: Se trata de todas las variables definidas en la clase y denominadas **variables de clase**.

De esta manera, tenemos una estructura jerárquica en la que aparecen, en primer lugar, las variables de clase; a continuación, las variables locales, y por últimos, las variables de bloque, como se muestra en la siguiente figura 12:

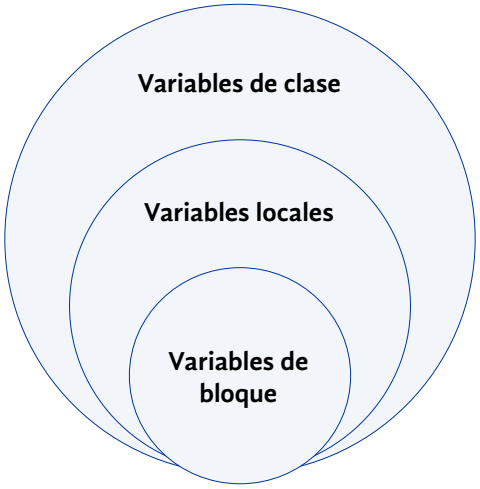

Fig.12. Jerarquía en el ámbito de uso de las variables.

En el siguiente ejemplo podemos ver el ámbito de los tres tipos de variables:

```
Class Ambito {
        int varClase;
        ...
        void método() {
                int varLocal;
                ...
                while(){
                        intvarBloque;
                        ...
                }
        }
}
```

Si se analiza el código, podemos observar que:

- La variable varClase puede ser utilizada en todo el código.
- La variable varLocal puede ser utilizada dentro del método pero fuera del método no existe.
- La variable varBloque únicamente puede ser empleada dentro del bucle, ya que fuera del bucle no existe.

/ 11. Ocultación de atributos

Ya hemos visto que existen distintos ámbitos en el uso de variables, pudiendo distinguir los siguientes:

- **Ámbito de la clase**.
- **Ámbito del método**.
- **Ámbito local**.

Aunque dos variables pertenezcan a ámbitos distintos, si se trata de ámbitos anidados, no pueden tener el mismo identificador, ya que se generaría un error de sintaxis. Sin embargo, existe una excepción al emplear una variable local a un método sobre una variable de clase. Solo en este caso, la variable local tiene prioridad sobre la variable de clase. En POO se dice que la variable local **oculta** a la variable de clase.

Veamos un ejemplo:

```
12  public class Persona {
13      String nombre;
14      int edad;
15      String apellido;
16
17
18      public void cumplirAños(){
19                      Local variable hides a field
20                      ----
21                      (Alt-Enter shows hints)
22          double edad= 3;
23          System.out.println("Tienes "+edad + " años");
24      }
```

Fig.13. Ocultación de atributos.

La única manera de impedir la ocultación de atributos es mediante el uso de la palabra reservada *this*. Solo con ella se puede emplear una variable definida que haya sido previamente ocultada al tener el mismo identificador.

La palabra this hace referencia a la propia clase, al objeto actual, y de esa manera permite acceder a los atributos que hayan sido ocultados.

Vamos a ver el ejemplo anterior (figura 13) mediante el uso de la palabra reservada this:

```
12  public class Persona {
13      String nombre;
14      int edad;
15      String apellido;
16
17
18      public void cumplirAños(){
19                      Local variable hides a field
20                      ----
21                      (Alt-Enter shows hints)
22          double edad= 3;
23          System.out.println("Tienes "+edad + " años");
24          this.edad = 32;
25      }
```

Fig.14. Uso del this.

En el ejemplo de esta figura 14 se puede observar como en la línea 22 se sigue marcando que oculta un campo, mientras en la línea 24, al utilizar la palabra this, se marca la variable en verde, de la misma manera que aparece en la línea 14 al declararla como variable de clase.

/ 12. Caso práctico 2: "Shadowing de variables"

Planteamiento: El *shadowing* de variables es un concepto, en programación, que ocurre cuando una variable en un ámbito más interno (como dentro de un método o un bloque) tiene el mismo nombre que una variable en un ámbito más externo (como una variable de instancia o una variable local en un ámbito externo). Cuando esto sucede, la variable en el ámbito más interno 'somete' u 'oculta' a la variable en el ámbito más externo, haciendo que solo la variable en el ámbito más interno sea accesible dentro de ese ámbito específico.

Nudo:

```java
public class EjemploShadowing {
    private int base = 10; // Variable de instancia

    public void metodoEjemplo() {
        int base = 20; // Variable local que hace shadowing a la variable de instancia

        System.out.println("Valor de base dentro del método: " + base);
        System.out.println("Valor de base fuera del método: " + base);
    }

    public static void main(String[] args) {
        EjemploShadowing ejemplo = new EjemploShadowing();
        ejemplo.metodoEjemplo();
    }
}
```

Fig.15. Ocultación de variables.

Dado el código de la figura 15, se quiere imprimir la variable de instancia con valor 10 y la variable local con valor 20. ¿Es posible con el código mostrado? En caso contrario, ¿cuál sería la solución?

Desenlace:

```java
public class EjemploSinShadowingCambioDeNombre {
    private int baseInstancia = 10; // Variable de instancia

    public void metodoEjemplo() {
        int baseLocal = 20; // Variable local que NO hace shadowing a la variable de instancia

        System.out.println("Valor de base dentro del método: " + baseLocal);
        System.out.println("Valor de base fuera del método: " + baseInstancia);
    }

    public static void main(String[] args) {
        EjemploShadowing ejemplo = new EjemploSinShadowingCambioDeNombre();
        ejemplo.metodoEjemplo();
    }
}


public class EjemploSinShadowingUsoThis {
    private int base = 10; // Variable de instancia

    public void metodoEjemplo() {
        int base = 20; // Variable local que NO hace shadowing a la variable de instancia

        System.out.println("Valor de base dentro del método: " + base);
        System.out.println("Valor de base fuera del método: " + this.base);
    }

    public static void main(String[] args) {
        EjemploShadowing ejemplo = new EjemploSinShadowingUsoThis();
        ejemplo.metodoEjemplo();
    }
}
```

Fig.16. Solución a la ocultación de variables.

Para evitar el shadowing y asegurarte de que estás utilizando la variable correcta, se pueden seguir estos pasos:

- Usar nombres de variables claros, **únicos** y descriptivos para evitar conflictos de nombres.
- Emplear la palabra reservada this para referirse a variables de instancia.
- Evitar declaraciones de variables redundantes con el mismo nombre en un ámbito más interno cuando no es necesario.

Por tanto, una posible solución al código sería el mostrado en la figura 16.

/ 13. Paso de parámetros y método toString()

A la hora de invocar los métodos de los objetos y pasarles parámetros, existen dos técnicas: el paso por valor y el paso por referencia.

Cuando se utiliza el **paso por valor** en los parámetros de un método, se realiza una copia del valor original, por lo que si realizamos cualquier modificación de ese parámetro dentro del método, esta no tendrá efecto en la variable original.

Cuando se utiliza el **paso por referencia** en los parámetros de un método, se está enviando una referencia (dirección de memoria) de la variable original, por lo que, si realizamos cualquier modificación de ese parámetro dentro del método, se hará también en la variable original.

En Java, las variables primitivas (int, long, float, double, char y bool) se pasan por valor, mientras que los objetos se pasan siempre por referencia. Observemos la siguiente figura 17.

```
public static void main(String[] args) {
    Persona p = new Persona("Bruce", "Wayne");
    cambiar(p);
    System.out.println("Persona: " + p);    // tiene redefinido el método toString()
}

static void cambiar(Persona persona) {
    persona = new Persona("Peter", "Parker");
}
```

Fig.17. Paso de un objeto como parámetro.

Vídeo 1. "Paso por valor y por referencia"
https://bit.ly/2BX12dl

El **método toString()** es un método del que disponen todas las clases y que muestra un **'resumen' del contenido de las variables**.

El resultado del método toString deberá ser diferente en distintos objetos (a no ser que tengan los mismos valores en las variables).

El modo de formatear la información dentro del método toString se puede hacer libremente, no habiendo un estándar definido para ello. Se puede mostrar, además, todas las variables de la clase o solamente las que se deseen permitiendo incluso llamar a algún método.

Este método no recibirá ningún parámetro y devolverá siempre un dato de tipo String, donde estará la información con los valores de nuestro objeto formateados.

Siguiendo el ejemplo de la clase Coche, su método toString será el que viene indicado en la siguiente figura 18.

```
@Override
public String toString() {
    String mensaje = "Datos del coche:\n";
    mensaje += "\tMarca: " + marca;
    mensaje += "\n\tModelo: " + modelo;
    mensaje += "\n\tMatrícula: " + matrícula;
    mensaje += "\n\tPrecio: " + precio + "€";
    return mensaje;
}
public static void main(String[] args) {
    // Creamos un objeto de tipo coche
    Coche hyundaii30 = new Coche("Hyundai", "i30", "12345KLM", 16000);
    System.out.println(hyundaii30);
}
```

Fig.18. Método toString().

/ 14. Documentación de métodos

Además de las clases, otra cosa que debemos documentar son los métodos de las mismas.

Todos y cada uno de los métodos que desarrollemos en las clases se deberán documentar, ya sean funciones o procedimientos.

Los métodos 'especiales', por llamarlos así, como son los constructores por defecto y con parámetros, métodos *get*, métodos *set* y el método toString también deberán documentarse.

La forma de documentarlos será de igual forma mediante el **generador de documentación Doxygen**, concretamente entre /** y */.

Las etiquetas de las que disponemos para documentar los métodos son las siguientes:

- **@param**: Esta etiqueta indicará qué representa un parámetro del método. Solo se utilizará en caso de que el método tenga parámetros, y se tendrá que poner etiqueta *param* por cada parámetro.
- **@return**: Esta etiqueta indicará qué es lo que devuelve el método. Solo se utilizará en caso de que el método devuelva algo, es decir, sea una función.
- **@deprecated**: Se emplea solo cuando queramos indicar que el método está obsoleto.
- **@see**: Es necesaria en caso de que nuestro método llame a otro método, y es con la que indicaremos el nombre del método al que llamamos. Habrá que poner una por cada método que llamemos.
- **@throws**: Esta etiqueta indicará una excepción que lance nuestro método y mediante ella explicaremos el porqué de la excepción. Será necesario escribir una por cada excepción que se lance.

Fig.19. Documentación de métodos con Doxygen.

Vídeo 2. "Documentación de una clase y sus métodos"

https://bit.ly/2WgnWmF

/ 15. Resumen y resolución del caso práctico de la unidad

Este tema abarca la identificación de las características de un objeto, el ciclo de vida del mismo y su almacenamiento en memoria. Se destaca la habilidad para crear y utilizar objetos, junto con el manejo de sus atributos y métodos.

Además, se aborda la comprensión de los distintos ámbitos de variables y su visibilidad, así como la diferencia entre paso por valor y paso por referencia en la manipulación de datos, junto a cómo mostrarlos por pantalla gracias al método toString, crucial para representar objetos como cadenas de texto legibles.

Finalmente, se subraya la importancia de documentar los métodos de una clase con herramientas como Doxygen, facilitando la comprensión y el mantenimiento del código por parte de otros desarrolladores.

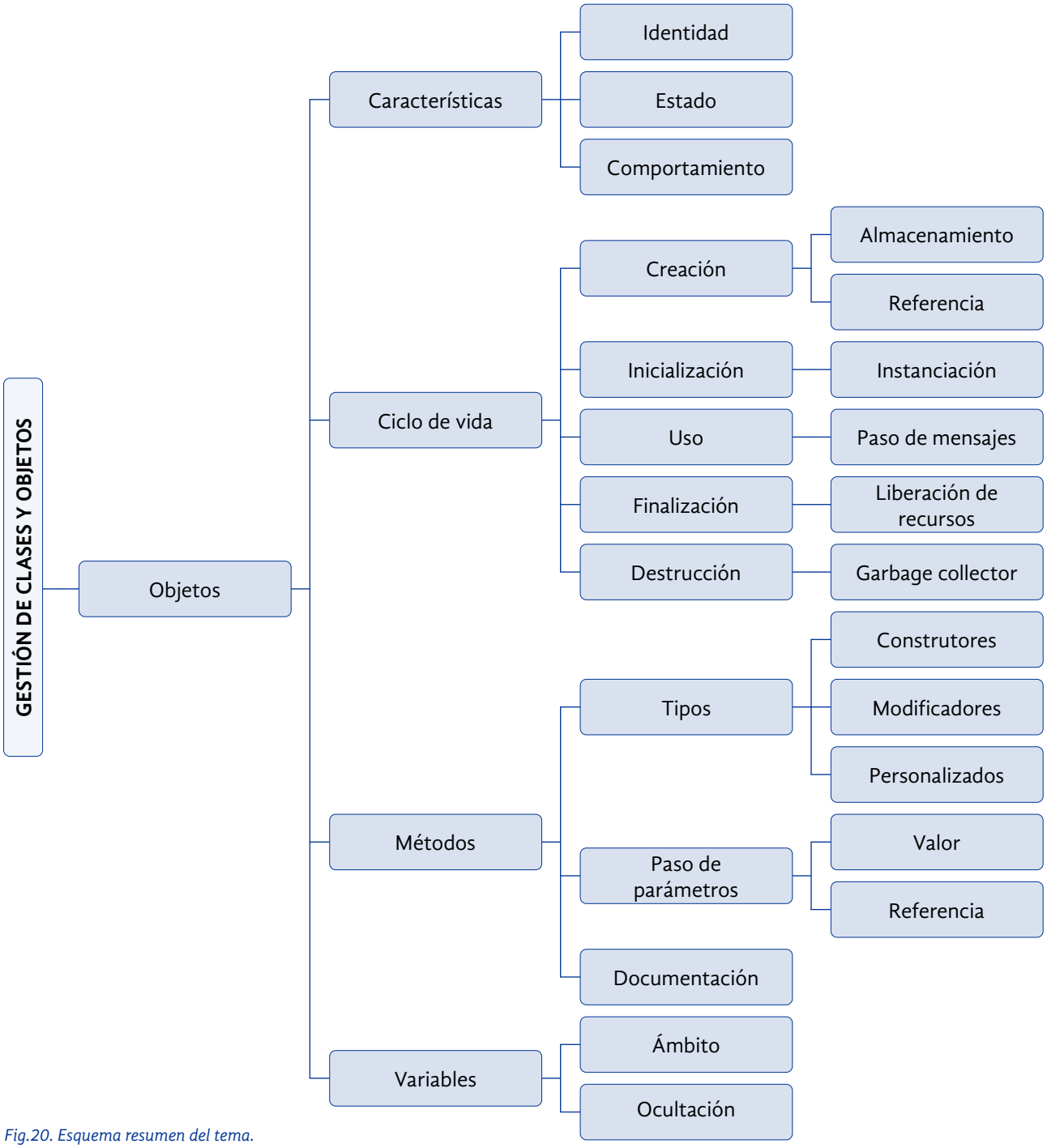

Fig.20. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Cuando se está resolviendo un problema en programación, independientemente del lenguaje que estemos usando, es conveniente modularizar lo máximo posible. Con la modularización se va a conseguir dividir un problema relativamente complicado de resolver en subproblemas más pequeños y, por ende, más fáciles de resolver.

Una buena práctica es tener pocas líneas en el método main. La idea es tener un main limpio y corto, más fácil de entender y mantener, para que los desarrolladores puedan rápidamente entender el flujo general del programa.

Además, al mover la lógica de negocios fuera del main, se promueve la separación de responsabilidades, lo que da como resultado un diseño más modular y desacoplado, para que el que es más fácil escribir pruebas unitarias y reutilizar el código en diferentes contextos.

/ 16. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos*. Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.